

INNEBYGDE DATASYSTEMER FOR MEKA- TRONIKK

Fullstendig prosjektrapport for MAS245 Innebygde Datasystemer for mekatronikk 2025

Lars Nøkland
Kristie Iselinn Storvik
Anastasia Volokhoff

SUPERVISOR
Kristian Muri Knausgård

University of Agder, 2025
Faculty of Engineering and Science
Department of Engineering and Sciences

Innhold

Figurer	iii
Tabeller	v
1 Introduksjon	1
2 Teori	2
2.1 Mikrokontrollere	2
2.1.1 ATmega168	2
2.1.2 Teensy 3.6	3
2.1.3 Mikroprosessorer	3
2.1.4 Raspberry Pi 3	3
2.2 Kommunikasjonsprotokoll	3
2.2.1 CAN-bus	4
2.3 Operativsystemer	4
3 Metode	5
3.1 Grunnleggende oppsett av mikrokontroller	6
3.1.1 Strømforsyning og spenningsregulator	6
3.1.2 Reset- og decoupling-krets	7
3.1.3 Beregning av seriemotstand for LED	8
3.1.4 PWM-styring av LED	8
3.1.5 Programmering og testing	8
3.2 Kommunikasjon mellom enheter	11
3.2.1 Oppsett av CAN-bus hardware	11
3.2.2 Implementering av CAN-kommunikasjon	12
3.2.3 Test av CAN-kommunikasjon: Ping-Pong-spill	14
3.3 Operativsystem og Raspberry Pi	15
3.3.1 Konfigurering og installasjon av operativsystem	15
3.3.2 Testing og verifisering av systemkobling	16
4 Resultater	17
5 Konklusjon	25
A SoftBlink kode	26
B PING-PONG kode for Teensy 3.6	28
Bibliography	37

Figurer

2.1	Utdrag fra tabell 33-3 i databladet til ATmega168	3
2.2	CAN logikk. Bilde hentet fra [5].	4
3.1	Strømforsyningskrets tegnet i KiCad.	6
3.2	Utdrag fra databladet til spenningsregulatoren [15].	7
3.3	Måler utgangsspenningen til strømforsyningskretsen.	7
3.4	skjematisk oversikt host/target	8
3.5	Pin-nummer for SPI kobling av Atmel ICE	9
3.6	Feilmelding ved valg av feil enhet	9
3.7	Signature bytes from ATmega datasheet[1]	10
3.8	CAN pluggen og hvilken pin som går til hvor	11
3.9	Koblingskjema for CAN	12
3.10	Koblingen til PEAK sin board, ikke loddet av denne gruppen	12
3.11	CAN test med PCAN-view	13
3.12	CAN test som viser sendte, motatte og ID-en til siste CAN melding.	13
3.13	PCAN-view som viser at CAN koblinge mellom PC-en og Raspberry PI3 funker	16
4.1	Kretskjema	18
4.2	CAN melding med PEAK	19
4.3	Zoomed it bilde av 4.3	19
4.4	To Teensy-enheter som starter å spille mot hverandre	21
4.5	To Teensy-enheter midt i et spill. Det hvite som er på skjermene er ballen som driver å spretter rundt	22
4.6	To Teensy-enheter hvor en har vunnet og den andre har tapt	22
4.7	Raspberry vs Teensy i Ping-Pong	23
4.8	Raspberry vs Teensy i Ping-Pong	23
4.9	Zoomed inn bilde av 4.9	23

Tabeller

3.1	Størrelse på lagringsmedier på ATmega168[1]	10
4.0	Gamma-korreksjonen	17
4.0	SoftBlink Loop	18
4.0	Sending av CAN meldinger	20
4.0	Motak av CAN meldinger	21

Kapittel 1

Introduksjon

Dette prosjektet har som mål å gi praktisk erfaring med mikrokontrollere/mikroprosessorer, programmering av grensesnitt, kommunikasjonsprotokoller samt styring og drivning av kretser, med et ønske om å utvikle en egen “datamaskin” med et egendefinert operativsystem.

Tre mikrokontrollere ble valgt for å utforske dette: ATmega168, Teensy 3.6 og Raspberry Pi 3. Som kommunikasjonsprotokoll ble CAN-bus benyttet, på grunn av dens enkle oppkobling og høye hastighet. CAN-bus tilbyr også adresserbare meldinger, inkludert muligheten til å prioritere dem. Protokollen er i tillegg mye brukt i ulike fagfelt, særlig innen bilsystemer og kjøretøykommunikasjon. Prosjektet går fra å sette opp en enkel krets med ATmega168 til å programmere og koble sammen et system som kan fungere som en enkel datamaskin.

Gjennom prosjektet ble det lagt vekt på å kombinere teori og praksis for å oppnå en helhetlig forståelse av hvordan maskinvare og programvare samhandler i et innebygd system. Målet er å opparbeide praktisk erfaring med innebygde datasystemer, og utvikle en bedre forståelse for hvordan slike systemer bygges opp og kommuniserer. Det ønskede læringsutbyttet etter dette prosjektet er å kunne sette opp en hvilken som helst krets med en mikrokontroller/mikroprosessor for å kunne gjøre diverse prosjekt, om det er å bygge en liten programmerbar krets eller om det blir et et kjørbart program på et eget operativsystem.

Kapittel 2

Teori

Dette kapitlet presenterer det teoretiske grunnlaget som danner rammen for prosjektet. Målet er å gi en nødvendig forståelse av de tekniske komponentene som inngår i systemet, slik at valg av maskinvare, spenningsregulatorer og kommunikasjonsløsninger kan begrunnes på et faglig solid grunnlag. Siden prosjektet benytter flere mikrokontrollere som gruppen ikke tidligere har arbeidet med, er det viktig å etablere et klart bilde av deres kapasitet, arkitektur og praktiske krav før implementasjonen. Teorien gir dermed innsikt i hvilke funksjoner enhetene tilbyr, hvilke begrensninger de har, og hvordan de må integreres i en krets for å fungere stabilt.

Kapitlet begynner med en gjennomgang av mikrokontrollere, inkludert ATmega168, Teensy 3.6 og Raspberry Pi 3. Disse beskrives med fokus på deres oppbygning, minnearkitektur og typiske bruksområder, som danner grunnlaget for videre designvalg i prosjektet. Deretter presenteres CAN-bus som kommunikasjonsprotokoll, og hvordan denne protokollen gjør det mulig å kommunisere mellom enheter på en strukturert og pålitelig måte. Samlet gir teorien et grunnlag som er nødvendig for metodene og implementasjonen som følger i senere kapitler.

2.1 Mikrokontrollere

En mikrokontroller er en integrert krets som inneholder prosessor (CPU), minne og ulike I/O-enheter på én chip. Denne integrasjonen gjør at mikrokontrollere kan kommunisere med sensorer og aktuatorer, behandle data og utføre instruksjoner i sanntid. De er spesielt godt egnet i innebygde systemer som er utviklet for å utføre spesifikke oppgaver som involverer styring eller overvåking av komponenter. [6]

2.1.1 ATmega168

ATmega168 er en 8-bit mikrokontroller fra Microchip Technology som baserer seg på AVR RISC-arkitekturen [1]. Den er utviklet for lavt strømforbruk, høy ytelse og enkel integrasjon i små innebygde systemer. Mikrokontrolleren bruker en modifisert Harvard-arkitektur, der programminne og dataminne er fysisk adskilt. Dette gjør at CPU-en kan hente instruksjoner fra flash samtidig som data behandles i SRAM, noe som gir effektiv og forutsigbar kjøring.

ATmega168 støtter en klokkehastighet på 20 MHz og tilbyr 16 KB flashminne for programlagring, 1 KB SRAM og 512 B EEPROM. Den har 23 programmerbare I/O-pinner, tre timer/tellere og innebygde kommunikasjonsgrensesnitt som SPI, I²C (TWI) og UART. Dette gjør den fleksibel i bruk og egnet for mange typer kretsdesign og styringsoppgaver.

Databladet oppgir en driftsspenning mellom 2.7–5.5 V, med et maks strømforbruk rundt 12 mA ved 8 MHz og 5 V driftsspenning, som vist i figur 2.1. Valg av spenning påvirker både effektforbruk og varmeutvikling i kretsen: en høyere spenning vil gi økt effekt. Lavere effekt gir mindre varmeutvikling i kretsen og dermed potensielt lengre levetid for komponentene. Samtidig vil en lavere

driftsspenning redusere maksimal oppnåelig klokkefrekvens.

33.2.1. DC Characteristics – Current Consumption

Table 33-3. DC characteristics - $T_A = -40^{\circ}\text{C}$ to 85°C , $V_{CC} = 1.8\text{V}$ to 5.5V (unless otherwise noted)

Symbol	Parameter	Condition	Min.	Typ. ⁽²⁾	Max.	Units
I_{CC}	Power Supply Current ⁽¹⁾	Active 1MHz, $V_{CC} = 2\text{V}$			0.55	mA
		Active 4MHz, $V_{CC} = 3\text{V}$			3.5	
		Active 8MHz, $V_{CC} = 5\text{V}$			12	
		Idle 1MHz, $V_{CC} = 2\text{V}$		0.25	0.5	
		Idle 4MHz, $V_{CC} = 3\text{V}$			1.5	
		Idle 8MHz, $V_{CC} = 5\text{V}$			5.5	
	Power-down mode ⁽³⁾	WDT enabled, $V_{CC} = 3\text{V}$		8	15	μA
		WDT disabled, $V_{CC} = 3\text{V}$		1	2	

Figur 2.1: Utdrag fra tabell 33-3 i databladet til ATmega168

2.1.2 Teensy 3.6

Teensy 3.6 er et utviklingskort basert på ARM Cortex-M4F-mikrokontrolleren MK66FX1M0VMD18 fra NXP. Kortet er designet for høy ytelse, lavt strømforbruk og fleksibel bruk i innebygde systemer. [9]

Mikrokontrolleren kjører på inntil 180 MHz og inkluderer 1 MB flashminne og 256 KB RAM. Teensy 3.6 tilbyr et stort antall I/O-pinner (58 totalt), hvorav flere støtter funksjoner som PWM, analoge innganger, SPI, I²C, UART og CAN-bus (via FlexCAN-modulen). Kortet inneholder også støtte for SD-kort, DAC-utganger, høyoppløselige ADC-er og USB 2.0 med høy hastighet. (Sjekk at dette er riktig med databladet)

På grunn av den høye ytelsen og det store antallet grensesnitt, brukes Teensy 3.6 ofte i prosjekter som krever rask databehandling, signalbehandling, sanntidskontroll og kommunikasjon mellom enheter.

2.1.3 Mikroprosessorer

I motsetning til en mikrokontroller, er en mikroprosessor kun en prosesseringsenhet som utfører instruksjoner og behandler data i en datamaskin eller annen digital enhet. [14].

2.1.4 Raspberry Pi 3

Raspberry Pi 3 Model B er et lite, enkeltkort-datasystem basert på mikroprosessen Broadcom BCM2837, en 64-bit ARM Cortex-A53 med fire kjerner og en klokkehastighet på 1.2 GHz. [12] I motsetning til en mikrokontroller kan Raspberry Pi kjøre et fullt operativsystem, typisk en Linux-distribusjon som Raspberry Pi OS. [11]

Kortet er utstyrt med 1 GB RAM, innebygd Wi-Fi og Bluetooth, samt et bredt utvalg av grensesnitt som GPIO-pinner, SPI, I²C, UART og USB-porter [13]. Dette gjør Raspberry Pi godt egnet som et mer avansert kontroll- og behandlingssystem i prosjekter som krever betydelig programlogikk, filsystem, nettverksstøtte eller sanntidsovervåkning.

I dette prosjektet ble Raspberry Pi 3 brukt for å utvikle og teste et enkelt operativsystemmiljø og for å kommunisere med andre enheter gjennom definerte grensesnitt.

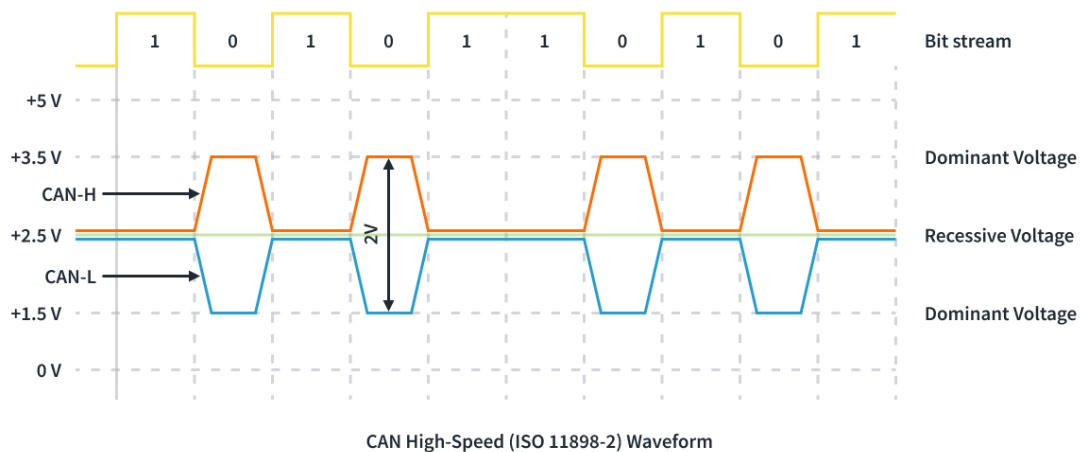
2.2 Kommunikasjonsprotokoll

En kommunikasjonsprotokoll er et sett med regler som styrer hvordan data blir overført mellom to eller flere enheter i et digitalt system. Protokollen definerer blant annet hvordan data struktureres, hvordan enheter identifiseres, hvordan feil håndteres og hvordan kommunikasjonen skal starte og avsluttes. Slike protokoller er nødvendige for at ulike komponenter skal kunne utveksle informasjon på en pålitelig og forutsigbar måte. [10]

2.2.1 CAN-bus

CAN, som står for *Controller Area Network*, er et balansert (differensielt) buss-interface der signalene sendes over to ledninger. Den logiske tilstanden på bussen bestemmes av spenningsdifferansen mellom ledningene CAN_H og CAN_L. [4] CAN bruker ikke-retur-til-null (NRZ) koding med “bit-stuffing” for datakommunikasjon, noe som gir effektive meldinger med få overganger og høy motstand mot støy. [4]

For at meldinger sendt fra noder ikke skal kollidere, blir de sortert etter ID ved å sammenligne bits. Dominant bit (0) kommer når CAN_H går høyt, og CAN_L går lavt, som vist på figur 2.2. Lave ID'er har flere dominante bits tidlig, og fordi det dominante bittet driver signalet lavt. Vil noder som prøver å sende recessivt bit (1) slutte grunnet forskjell mellom ønsket og faktisk verdi på porten.



Figur 2.2: CAN logikk. Bilde hentet fra [5].

2.3 Operativsystemer

Alle datamaskiner og innebygde systemer har et operativsystem, enten det er Linux, Windows, macOS eller et egendefinert system. Operativsystemet fungerer som et mellom mann mellom maskinvare og programvare, og styrer blant annet lasting og kjøring av programmer. [7]

Kapittel 3

Metode

Dette kapitlet beskriver den praktiske gjennomføringen av prosjektet, fra etablering av grunnleggende mikrokontrolleroppsett til integrasjon av kommunikasjonssystemer og utvikling av et eget operativsystem.

Arbeidet startet med ATmega168, der fokus lå på å forstå oppbygning og oppstart av en mikrokontroller på et helt grunnleggende nivå. Deretter beskrives implementasjonen av kommunikasjon mellom enheter ved hjelp av CAN-bus. Funksjonaliteten ble testet ved å lage et enkelt Ping-Pong-spill, hvor to enheter måtte utveksle data raskt nok til at posisjoner og hendelser ble oppdatert riktig. Dette gjorde det mulig å se om CAN-kommunikasjonen fungerte slik den skulle i praksis.

Avslutningsvis presenteres utviklingen av et tilpasset operativsystem for Raspberry Pi 3 ved bruk av Buildroot. Arbeidet innebar å velge riktig konfigurasjon i Buildroot, bygge operativsystemet, og få det til å fungere sammen med resten av prosjektet hvor både CAN-kommunikasjonen og programkoden skulle kjøres på Raspberry Pi.

3.1 Grunnleggende oppsett av mikrokontroller

Dette delkapitlet beskriver implementasjonen av en enkel mikrokontrollerkrets for ATmega168, med formål å forstå og demonstrere de sentrale kravene som må oppfylles for å ta mikrokontrolleren i bruk i en krets. Arbeidet omfatter utvikling av en egnet strømforsyningsløsning, implementering av nødvendige reset- og decoupling-funksjoner, samt tilrettelegging for programmering og grunnleggende I/O-styring. For å verifisere at kretsen fungerte som forventet, ble det implementert et enkelt testprogram i form av en «soft blink» på en LED. Programmet benyttet PWM og gammakorreksjon for å demonstrere både digital styring og sanntidskontroll av en utgang. Dette praktiske forsøket danner grunnlaget for de mer avanserte systemene som utvikles senere i rapporten.

3.1.1 Strømforsyning og spenningsregulator

Kretsen består av LED og mikrokontroller, der begge trekker lite strøm. Dermed ble en 3.3V spenningsregulator av type LF33AB et naturlig valg. De elektriske egenskapene for spenningsregulatoren er oppgitt i tabell i databladet [15]. For denne typen oppgis det en øvre grense for utgangsstrøm på 1A. Etter kalkulasjonene 3.1 - 3.4 gir dette en god margin i forhold til strømforbruket i kretsen. Strømforsyningskretsen ble koblet opp etter kretsen vist i figur 3.1, som ble basert på kretsen vist i figur 3.2 tatt fra databladet til strømforsyningen [15]. Det ble ikke brukt Inhibit-delen ettersom LF33AB alltid er aktiv

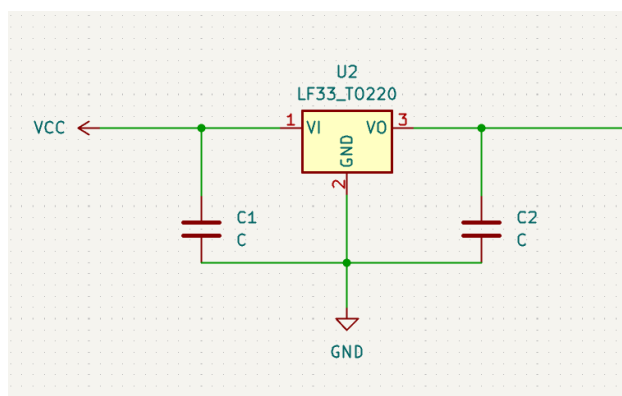
For å være på den sikre siden, tas det utgangspunkt i maksverdiene for strømforbruk for mikrokontroller $I_{ATmega} = 12mA$ og diode $I_{LED} = 10mA$ [8]. Kretsen vil da ha total effekt og strømforbruk:

$$I_{total} = I_{ATmega} + I_{LED} = 22mA \quad (3.1)$$

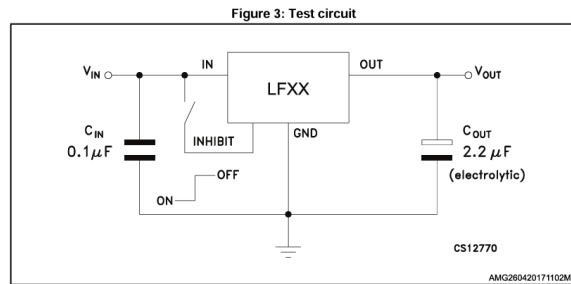
$$P_{total} = 3.3V \cdot 22mA = 72.6mW \quad (3.2)$$

$$I_{total,margin50\%} \approx 22mA \cdot 1.5 = 33mA \quad (3.3)$$

$$P_{total,margin50\%} \approx 3.3V \cdot 33mA = 109mW \quad (3.4)$$

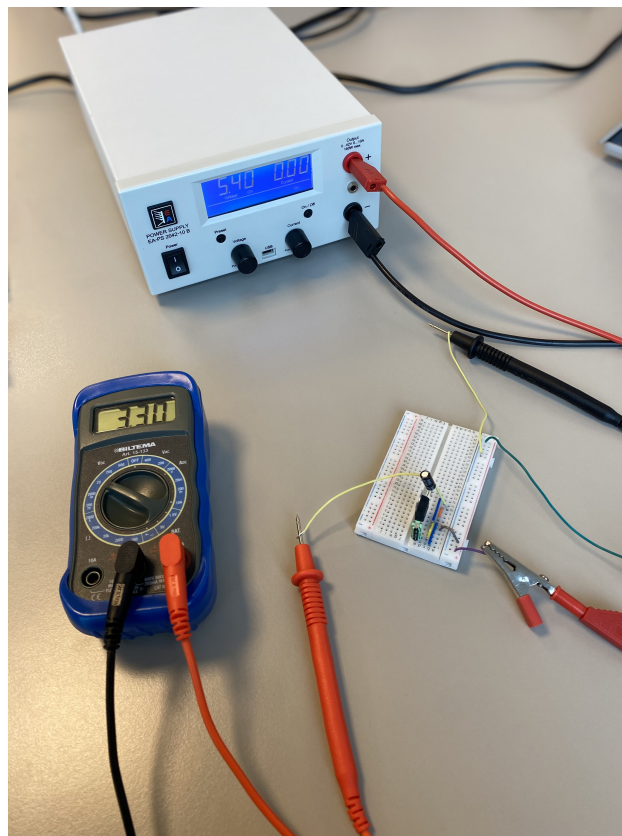


Figur 3.1: Strømforsyningskrets tegnet i KiCad.



Figur 3.2: Utdrag fra databladet til spenningsregulatoren [15].

Kretsen ble testet med lav inngangsspenning, og utgangsspenningen ble målt til 3,3 V, som forventet. Dette blir vist i figur 3.3. Her er det koblet en strømforsyning som sender inn en spenning på rundt 5V. Et multimeter er koblet i serie i kretsen og viser tydelig at spenningsregulatoren regulerer kretsen riktig og gir ut en spenning på 3.3V. Dette er det forventede resultatet som gjør at resten av kretsen kan bygges.



Figur 3.3: Måler utgangsspenningen til strømforsyningskretsen.

3.1.2 Reset- og decoupling-krets

I hardware-designet (figur 3.1, s. 7 i [3]) vises oppkoblingen av en reset-knapp til mikrokontrolleren. Dette ble koblet sammen med en fysisk push knapp" og en 330 Ω resistanse. Alt dette ble koblet i PIN 1 på mikrokontrolleren.

For å redusere støy på analoge signaler, ble det implementert decoupling kondensatorer til Vcc, Avcc og Aref. Databladet anbefaler en kondensator på 100nF, som også ble benyttet i dette designet (del 2.2 i [3]). Kondensatorene bidrar til å stabilisere spenningen og redusere støy på de analoge inngangene.

3.1.3 Beregning av seriemotstand for LED

Fra en generisk LED diode[8] finner vi at $10mA$ er en konservativ strømmengde, da dette er blant det lavere som blir oppgitt. Det vil derfor sikre sikker drift av systemet, uten å belaste komponentene unødvendig. Ved bruk av en rød LED-diode brukes $V_{led} = 2V$ [8].

Utrekning av diode motstand er gjort i eq.3.5.

$$\frac{V_{CC} - V_{LED}}{R} = I = \frac{3.3V - 2V}{R} \rightarrow \frac{3.3V - 2V}{10mA} = 130\Omega \quad (3.5)$$

Dette er også innenfor maks rating oppgitt i datablad for ATmega168[1] for IO-pins på $40mA$.

3.1.4 PWM-styring av LED

For å oppnå soft blink av LEDen, blir LEDen flyttet til en pin som har PWM-utgang på ATmega168, som typ PD6. LED-en er koblet slik at mikrokontrolleren synker strøm gjennom den. Soft blink implementeres ved hjelp av gammakorreksjon, som sørger for at lysstyrken oppfattes jevnt av det menneskelige øyet, til tross for lineær endring i duty cycle.

I databladet til LEDen finner vi maximum ratings for Peak forward current på $75mA$. Det betyr at LEDen tåler høyere strøm ved korte pulser og lav duty cycle. Dette gjør det mulig å bruke PWM uten å overskride LEDens maksimale kontinuerlige strøm.

For å velge PWM-frekvens brukes formelen:

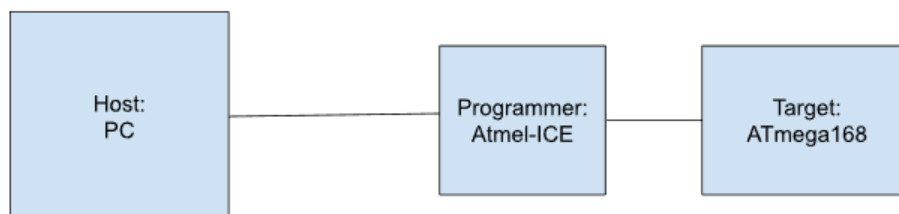
$$f_{PWM} = \frac{F_{CPU}}{prescaler \cdot 2^8} \quad (3.6)$$

Der F_{CPU} er klokkefrekvensen til mikrokontrolleren, og prescaler velges for å sikre en frekvens som gir stabil og synlig PWM-effekt uten å introdusere synlig flimmer.

3.1.5 Programmering og testing

Side 21 i ICE-databladet viser hvordan koblingen skal gjøres med SPI-grensesnittet [2].

Sammenkoblingen mellom datamaskinen og ATmega168 skjer gjennom en Atmel-ICE, som bruker SPI-kommunikasjon[2]. En ferdig nummerert *squid kabel* benyttes for oppkoblingen etter tabelloversikten i figur 3.5.



Figur 3.4: skjematisk oversikt host/target

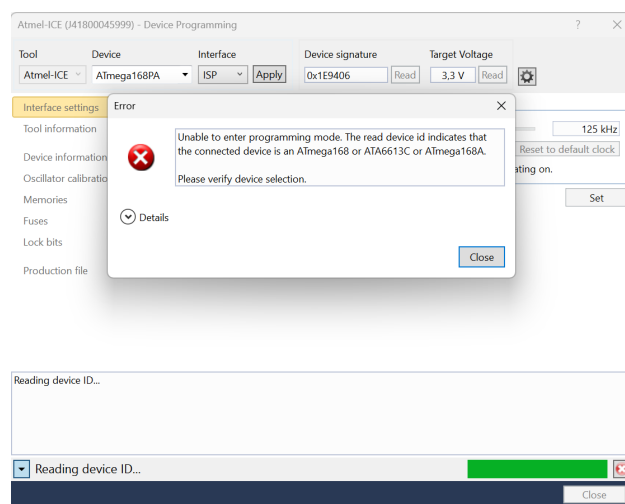
Table 3-6. Atmel-ICE SPI Pin Mapping

Atmel-ICE AVR port pins	Target pins	Mini-squid pin	SPI pinout
Pin 1 (TCK)	SCK	1	3
Pin 2 (GND)	GND	2	6
Pin 3 (TDO)	MISO	3	1
Pin 4 (VTG)	VTG	4	2
Pin 5 (TMS)		5	
Pin 6 (nSRST)	/RESET	6	5
Pin 7 (not connected)		7	
Pin 8 (nTRST)		8	
Pin 9 (TDI)	MOSI	9	4
Pin 10 (GND)		0	

Figur 3.5: Pin-nummer for SPI kobling av Atmel ICE

Når koden skal bygges og lastes inn på mikrokontrolleren benyttes Microchip Studio, ettersom denne IDE-en har ferdige drivere som forenkler kommunikasjon og programmeringen. Det blir gjort en krysskompilering inne i Microchip Studio etter som pc-en som blir brukt har en x86-arkitektur mens ATmega168 bruker en AVR-arkitektur. Enkelt så betyr dette at programvaren bygges på en host med en arkitektur for å så kunne kjøre på en annen arkitektur. Derfor kan ikke PC-en kjøre koden direkte, men må fungere som et typisk utviklingsmiljø og programmeringsverktøy.

Når Microchip Studio settes opp for programmering med ATmega168, blir det sjekket at programmeren og mikrokontrolleren faktisk kommuniserer. Dette blir gjort ved å få programmet til å "Read Device Signature" og sammenligne dette med det som står i databladet for mikrokontrolleren. Tabellen som viser signatur coden er vist i 3.7. Inne i Microchip Studio er det da essensielt å velge riktig versjon av ATmega168, ellers oppstår det en feil som vist i figur 3.6.



Figur 3.6: Feilmelding ved valg av feil enhet

Koden skrives direkte til *Flash*-minnet på mikrokontrolleren slik at mest mulig av *SRAM*, og *EEPROM* er ledig. Utregning av gamma korreksjon blir også regnet ut på forhånd og lagret i *Flash* slik at dette ikke må beregnes under kjøring. Dette fører til en generelt effektiv bruk av den relativt begrensede maskinkraften som er til stede på denne mikrokontrolleren. Størrelsene på de forskjellige lagringsmediene er vist i tab.3.1 .

32.3. Signature Bytes

The device have a three-byte signature code. This code can be read in both serial and parallel mode, also when the device is locked. The three bytes reside in a separate address space. For the device the signature bytes are given in the following table.

Table 32-9. Device ID

Part	Signature Bytes Address		
	0x000	0x001	0x002
ATmega48/V	0x1E	0x92	0x05
ATmega88/V	0x1E	0x93	0x0A
ATmega168/V	0x1E	0x94	0x06

Figur 3.7: Signature bytes from ATmega datasheet[1]

Flash (Bytes)	16K
SRAM (Bytes)	1K
EEPROM (Bytes)	512

Tabell 3.1: Størrelse på lagringsmedier på ATmega168[1]

Erfaringen fra ATmega168 eksperimenteringen dannet grunnlaget for å implimentere kommunikasjoner mellom enheter via CAN-bus, som er et naturlig neste steg i dette prosjektet.

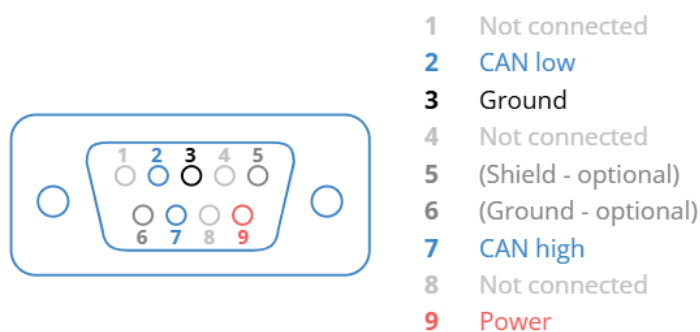
3.2 Kommunikasjon mellom enheter

Dette delkapitlet beskriver implementasjonen av et mikrokontrollerbasert system med fokus på Teensy 3.6, med formål å forstå og demonstrere hvordan denne kan benyttes i et innebygd system med CAN-bus. Arbeidet omfatter oppsett av Teensyen, tilrettelegging for programmering, samt grunnleggende kommunikasjon over CAN-bus. Etter å ha implementert og testet koding av CAN-bus-meldinger, vil systemet bli videreutviklet til å kjøre et enkelt spill som benytter CAN-bus-protokollen som kommunikasjonskanal. Dette praktiske forsøket danner grunnlaget for mer komplekse applikasjoner og viser hvordan høytytende mikrokontrollere kan integreres i sammensatte innebygde systemer.

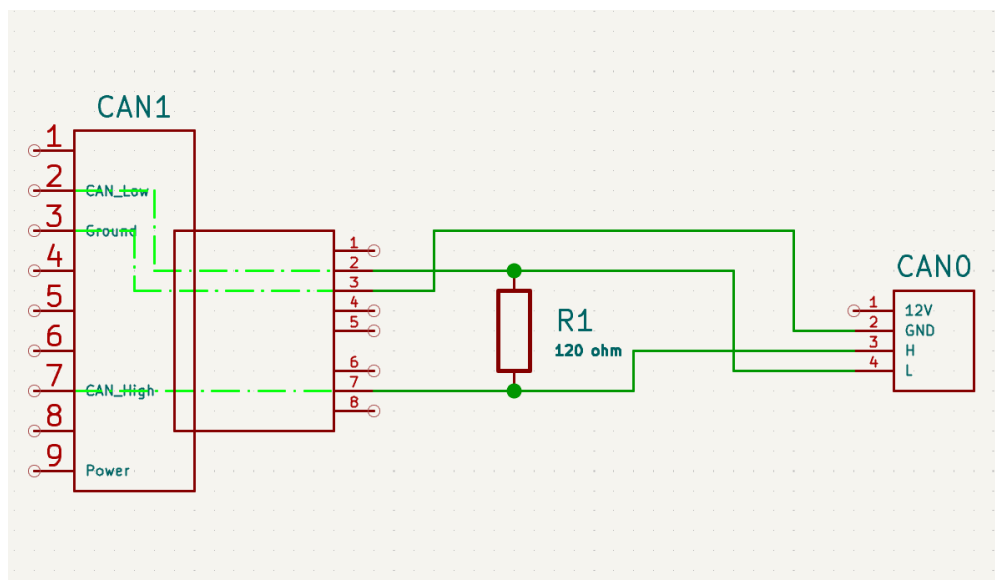
3.2.1 Oppsett av CAN-bus hardware

På Teensy 3.6 er CAN0 koblet til PIN 3 og PIN 4, mens CAN1 er koblet til PIN 33 og PIN 34. CAN0 fungerer som den primære CAN -utgangen, mens CAN1 er den sekundære.

Det ble funnet en eksisterende CAN-bus-tilkobling med en D-sub-adapter for PEAK-adapteren, som ble kontrollert og justert for å sikre korrekt funksjon. Et bilde som viser hvordan D-sub er koblet, vises i 3.10. Før bruk ble det kontrollert med multimeter at det ikke var kortslutning mellom CANH, CANL, VCC og GND. For å terminere bussen og redusere støy, ble en 120 Ω motstand koblet mellom CANL og CANH. Ground var allerede koblet, og det ble ikke lagt til strøm via CAN-bussen, da dette ikke er nødvendig for drift. Siden kun CAN0 ble benyttet, ble terminering kun kontrollert og implementert for denne. Dersom CAN1 skulle tas i bruk, måtte tilsvarende sjekk og eventuelt terminering gjennomføres. Et koblingsskjema som viser hvordan PEAK-kortet er koblet til Teensyen, vises i figur 3.9



Figur 3.8: CAN pluggen og hvilken pin som går til hvor



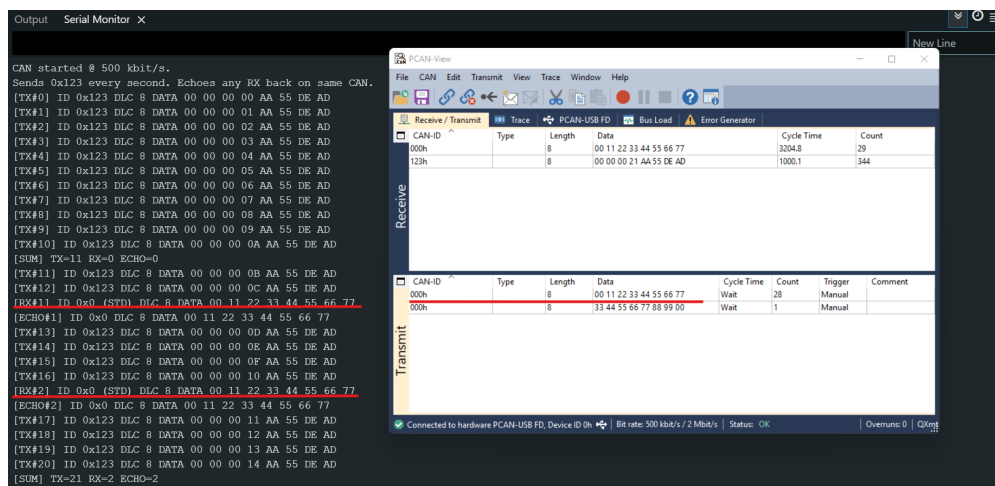
Figur 3.9: Koblingskjema for CAN



Figur 3.10: Koblingen til PEAK sin board, ikke loddet av denne gruppen

3.2.2 Implementering av CAN-kommunikasjon

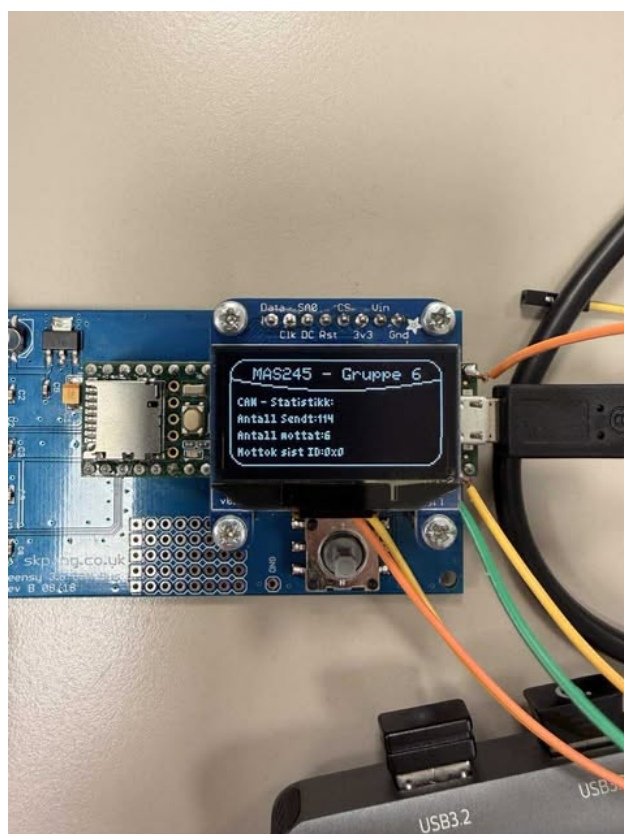
For å kunne teste at CAN kommunikasjonen fungerte ble dette testes med å koble PC-en og Teensy-en med en PCAN, og kjøre en Arduino-kode inn på Teensy 3.6. Koden sender meldinger som kan avleses i PCAN-View, samtidig som PCAN-View kan sende meldinger tilbake til Teensyen. Mottatte meldinger vises på Serial Monitor i Arduino IDE, forutsatt at korrekt kode er implementert på mikrokontrolleren.



Figur 3.11: CAN test med PCAN-view

For at dette skulle fungere, må et CAN-bibliotek initialiseres i Arduino. Her ble *flexcan_{t4}* brukt. En bitrate må settes med en `.setBaudRate`. Et meldingsobjekt ble laget som gjør det enklere å skrive flere type meldinger. Meldingsdata fylles med en `msg.buf[]`, mens `write` brukes for å skrive denne meldingen.

Testen ble videre utvidet med en mer kompleks kode. Her ble det ønsket å overvåke hvor mange CAN-meldinger Teensy 3.6 hadde sendt og mottatt, samt ID-en til den siste meldingen. Informasjonen ble kontinuerlig oppdatert og vist på Teensy sin OLED skjerm for å sikre sanntidsavlesning og verifisere korrekt kommunikasjon.



Figur 3.12: CAN test som viser sendte, motatte og ID-en til siste CAN melding.

3.2.3 Test av CAN-kommunikasjon: Ping-Pong-spill

Teensy 3.6 var allerede koblet til en OLED-skjerm og en joystick, så et naturlig steg for å teste CAN-kommunikasjonen var å bygge et spill. Ping-Pong ble valgt som test. Systemet er designet slik at to Teensyer kan kobles sammen via CAN, hver med sin skjerm og joystick.

Hver joystick styrer sin egen plate på skjermen, mens ballen kontrolleres av den Teensyen som er satt som master. Ballen spretter tilbake når den treffer en av platene, eller når platen treffer skjermens lange sider. CAN-meldingene som sendes kontinuerlig mellom Teensyene:

- Hver Teensy sender sin plateposisjon til den andre Teensyen.
- Master-Teensy sender i tillegg ballens posisjon og oppdaterer score.
- Mottaker-Teensy oppdaterer sin skjerm basert på mottatte data, inkludert speilvendte koordinater for ball og motstanderplate.

Utfordringer inkluderte å sikre at begge Teensyene ikke ble satt som master samtidig, korrekt speilvending av koordinater på mottakerskjermen, og å opprettholde tilstrekkelig oppdateringsfrekvens for jevn spilling.

Dataene som ble sendt mellom Teensyene ble pakket i bytes for å effektivt utnytte CAN-bussen. Plateposisjoner ble representert med én byte, siden platen kun beveger seg i y-retning, mens ballens posisjon ble sendt med to bytes for å representere både x- og y-koordinater.

Unike CAN-IDer ble definert for hver meldingstype, for å unngå konflikt og sikre at mottaker kunne identifisere data korrekt:

- Plateposisjon spiller 1: ID = 26
- Plateposisjon spiller 2: ID = 27
- Ballposisjon: ID = 56
- Rolle-claim (master/slave-logikk): ID = 16
- Game over (poengmelding): ID = 57
- Reset game (signal for restart): ID = 58

Ved oppstart ble én Teensy valgt som master, mens den andre startet som slave. Dette sikrer at ballen kun styres fra master, og at score og game state kun oppdateres fra en kilde.

Etter å ha demonstrert hvordan CAN-bus-kommunikasjonen kan brukes til å styre et ping-pong-spill, er neste steg å utvide systemet til å kommunisere med en Raspberry Pi 3 som kjører eget operativsystem. Dette åpner for utvikling av mer komplekse programmer og innebygde systemer, og fungerer som et naturlig overgangspunkt til det påfølgende kapittelet om operativsystem og integrasjon med Raspberry Pi.

3.3 Operativsystem og Raspberry Pi

Dette delkapitlet beskriver implementasjonen av et tilpasset operativsystem for Raspberry Pi 3, med formål å demonstrere hvordan Buildroot kan benyttes for å lage et system som støtter CAN-bus-kommunikasjon. Arbeidet omfatter konfigurering og installasjon av operativsystemet, tilrettelegging for programmering, samt utvikling av et enkelt program som muliggjør kommunikasjon via CAN-meldinger. Dette praktiske forsøket danner grunnlaget for videre integrasjon av Raspberry Pi med Teensy 3.6 og utvikling av mer komplekse innebygde systemer.

3.3.1 Konfigurering og installasjon av operativsystem

Arbeidet med Raspberry Pi 3 startet med testing av raspberrypi3-defconfig i Buildroot, som er en av mange maler som er tilgjengelig å bruke for Raspberry Pi3. Ved oppstart ble prosessorarkitekturen satt til i386, mens arkitekturvarianten var i586. Dette viste hvordan Buildroot identifiserer og setter standard parametere for CPU og maskinvare, selv om denne konfigurasjonen ikke var optimal for Raspberry Pi 3.

For å forenkle prosessen og sikre korrekt oppsett av systemet, ble en ferdig konfigurasjonsmappe levert av veileder benyttet. Denne mappen inneholdt alle nødvendige innstillinger for Raspberry Pi 3, inkludert korrekt prosessorarkitektur, CPU-kjerne og støtte for tilhørende maskinvare. Konfigurasjonen sikret kompatibilitet med ARMv8-A arkitektur (Cortex-A53) og 64-bit (AArch64), som er standard for Raspberry Pi 3.

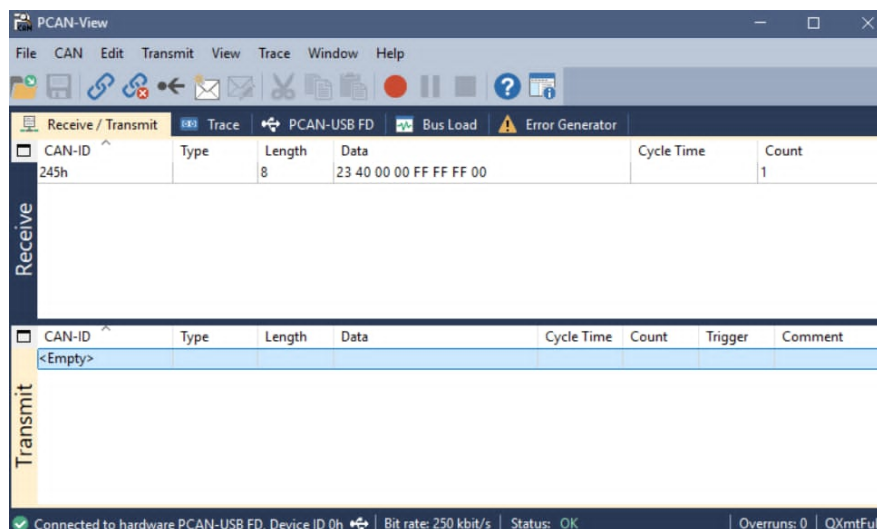
Som operativsystem ble en LTS-versjon (Long Term Support) valgt for både Ubuntu og Buildroot. LTS-versjoner er utviklet for stabilitet og langvarig support, noe som reduserer risikoen for feil og gir tilgang til oppdateringer og hjelp dersom problemer oppstår.

Raspberry Pi 3 benytter en Cortex-A53 CPU-kjerne, som implementerer ARMv8-A arkitektur med 64-bit støtte (AArch64)[12]. ARM er en prosessorfamilie mye brukt i innebygde systemer, ettkorts-datamaskiner og mobile enheter. Systemet er little endian, som betyr at den minst signifikante byen lagres først i minnet.

Bruk av veilederens ferdige konfigurasjonsmappe gjorde det mulig å generere et komplett og robust operativsystem uten omfattende manuell justering. Denne konfigurasjonen danner grunnlaget for videre testing og kobling mot prosjektets øvrige komponenter, inkludert CAN-bus og mikrokontrollere.

For å spesifisere hvor Buildroot genererer filer og bygger systemet, brukes parameteren `O=/filsti/til/output-folder`. Dette gjør det enklere å organisere filer, og ved å skrive `O=PWD` legges output direkte i den nåværende katalogen. For å bruke egne pakker og konfigurasjoner uten å endre Buildroots standardstruktur, kan `BR2 - EXTERNAL` settes til en ekstern katalog som inneholder egne defconfig-filer og patches. Dette er nyttig for å legge til spesialkonfigurasjoner for Raspberry Pi 3 uten å endre standard Buildroot-oppsett.

Operativ systemet og Raspberry Pi 3 ble testet med å gjøre en enkel CAN-bus test. En PC med PCAN-View ble koblet via CAN til Raspberry Pi 3. Raspberry Pi 3 fikk da sendt en melding med `cansend can0 245# 23.40.00.00.FF.FF.FF.00`, som ble lest av PCAN-view. For å teste at Raspberry Pi 3 også kunne motta meldinger, ble en melding sendt fra PCAN, og mottaket ble kontrollert ved hjelp av `candump can0`.



Figur 3.13: PCAN-view som viser at CAN koblinge mellom PC-en og Raspberry PI3 fungerer

3.3.2 Testing og verifisering av systemkobling

For å teste funksjonaliteten til det nye operativsystemet, ble det besluttet å bruke Ping-Pong-spillet som tidligere var utviklet. Spilletts logikk ble implementert i en kjørbare fil på Raspberry Pi 3, og systemet kommuniserte med en Teensy 3.6 for å motta signaler fra en joystick og for å tegne spillets grafikk på skjermen.

Siden spillet allerede var utviklet, var implementeringen relativt enkel. Spilletts logikk ble plassert i en .cpp-fil som ble kompilert til en kjørbare fil ved hjelp av Buildroot. Samtidig ble det utviklet en enkel Arduino-kode for Teensy 3.6, som muliggjorde sending og mottak av CAN-meldinger. Dette var nødvendig for å synkronisere spillets hendelser, inkludert tilbakestilling av skjermen ved start av nytt spill, og for å sikre korrekt kommunikasjon mellom mikrokontrolleren og Raspberry Pi 3.

Kapittel 4

Resultater

Prosjektet har hatt som mål å oppnå en dypere forståelse av mikrokontrollere og mikroprosessorer, med særlig fokus på kommunikasjon og sanntidskontroll. Det har blitt oppnådd flere vellykkede eksperimenter og forsøk med både CAN-bus kommunikasjoner, spill og kretsregulering.

ATmega168 ble koblet opp som en enkel mikrokontrollerkrets med de nødvendige komponentene som trengtes. Det har blitt koblet en strømforsyningskrets, dioder, kondensatorer for decoupling og reset switcher. Til tross for ønsket verdi på 130Ω ble en motstand på 150Ω brukt på grunn av tilgjengelighet, og kretsen fungerte stabilt. Dette ga en pålitelig basis for videre testing. Et fullt kretskjema kan bli sett i 4.1.

Målinger viste at spenningsregulatoren ga en stabil 3.3 V forsyning uten merkbare fluktuasjoner, noe som bekrefter at kretsen var egnet som grunnlag for videre eksperimenter. Med en veljustert spenningsregulator, hadde ATmegaen en stabil driftspenning. Videre ble en LED-diode koblet og SoftBlink ble implementert for å undersøke hvor presist lysstyrken kunne styres med PWM. SoftBlink fungerte som forventet, og gamma-korreksjonen ga en visuelt jevn og lineær opplevd endring i lysstyrke. Kodesnutt av både gamma-korreksjonen og loopen som kjører SoftBlink er vist i listing 1 og listing 2

```
// More aggressive gamma table (2.8 gamma) - spends less time at top ...
brightness
const uint8_t gamma_table[64] PROGMEM = {
    0,  0,  0,  0,  1,  1,  2,  2,
    3,  4,  5,  6,  7,  8, 10, 11,
    13, 15, 17, 19, 21, 23, 26, 28,
    31, 34, 37, 40, 43, 47, 50, 54,
    58, 62, 66, 70, 75, 79, 84, 89,
    94, 99, 104, 110, 115, 121, 127, 133,
    139, 145, 152, 158, 165, 172, 179, 186,
    193, 201, 208, 216, 224, 232, 240, 248
```

Listing 1: Gamma-korreksjonen

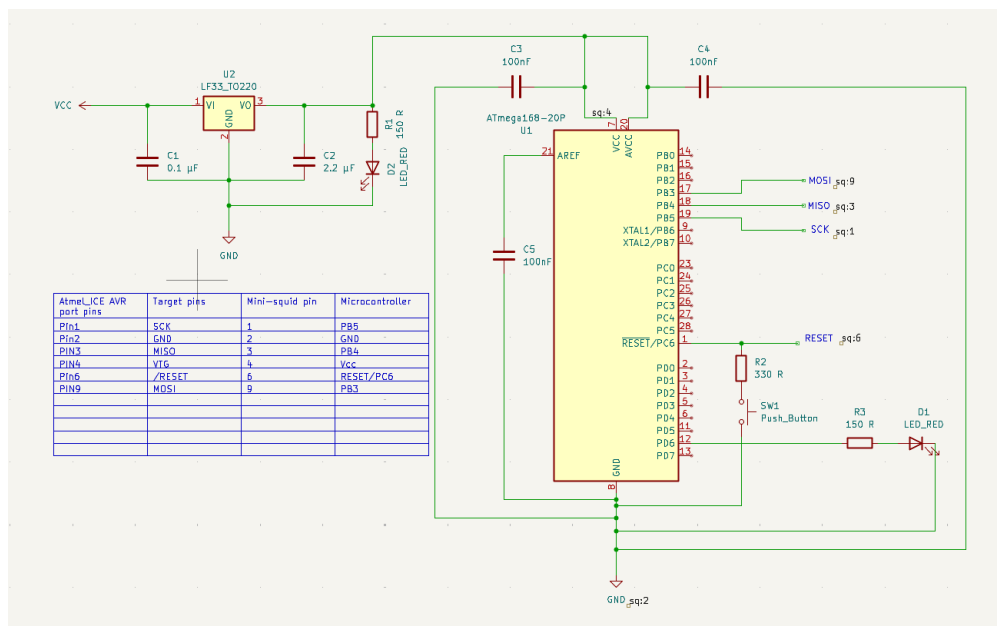
```

void soft_blink_loop(void) {
    // Fade in
    for (uint16_t i = 0; i < 256; i++) {
        set_gamma_corrected_brightness(i);
        _delay_ms(56);
    }

    // Fade out
    for (uint16_t i = 255; i > 0; i--) {
        set_gamma_corrected_brightness(i);
        _delay_ms(56);
    }
}

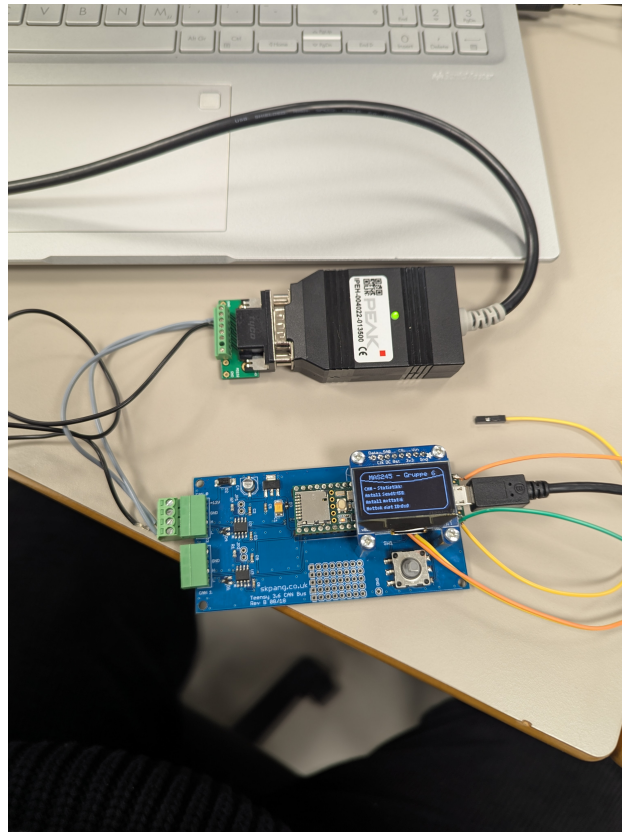
```

Listing 2: SoftBlink Loop

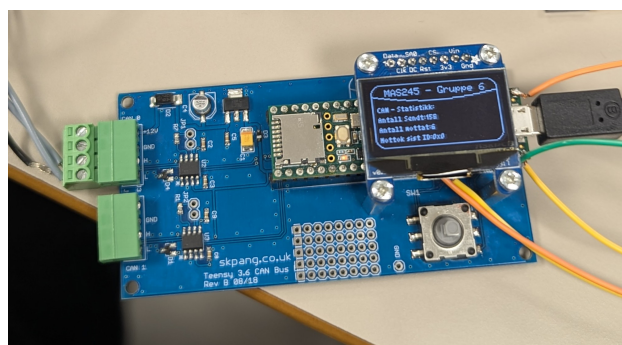


Figur 4.1: Kretskjema

Videre ble et mer komplisert system brukt for å teste kommunikasjonsprotokoller. En Teensy 3.6 ble koblet til en joystick og en OLED-skjerm, og hadde muligheten til å kommunisere med andre Teensyer via CAN-bus. Den første store testen viste at det er muligheter å sende tekst og tegninger med CAN-meldinger, og at disse meldingene blir sendt fort.



Figur 4.2: CAN melding med PEAK



Figur 4.3: Zoomed it bilde av 4.3

Ved en bitrate på 500 kbit/s ble ingen pakketap observert under testperioden 4.3, og prosjektet gikk videre til å implementert et Ping-Pong spill som kunne bli spilt med andre Teensy-enheter med hjelp av CAN.

Koden ble skrevet i Arduino og implementert i to Teensy-enheter som var koblet sammen med en CAN. Hver Teensy sendte sine plateposisjoner og master sendte ballposisjoner, med unike CAN-IDer for hver melding. Data ble pakket i bytes (1 byte for plateposisjon, 2 bytes for ballposisjoner), og systemet håndterte sanntidsoppdatering av spillobjekter stabilt, og forsøket vurderes som vellykket. Figurene ??, 4.5 og 4.6 viser oppsettet og resultatene av denne testen. Under vises to kode snutt fra Arduino koden som viser hvordan programmet mottar og sender CAN-meldinger. Hele koden er lagt ved i appendix 5

```

*/
void receiveCANMessages() {
    CAN_message_t rxMsg;
    while (Can0.read(rxMsg))
    {
        // 1. Rolleclaim
        if (rxMsg.id == idRoleClaim && rxMsg.len >= 1 && ...
            !isPlayerAssigned) {
            if (rxMsg.buf[0] == 1) {
                isPlayer2 = true;
                isPlayerAssigned = true;
                Serial.println("Jeg ble Player 2!");
            }
        }

        // 2. Plateposisjon fra motpart
        const int rxPlateId = isPlayer2 ? idPlatePositionP1 : ...
            idPlatePositionP2;
        if (rxMsg.id == rxPlateId && rxMsg.len >= 1) {
            remotePlatePosition = rxMsg.buf[0];
        }

        // 3. Ballposisjon (Bare P2 mottar)
        if (isPlayer2 && rxMsg.id == idBallPosition && rxMsg.len == 2) {
            xBall = SCREEN_WIDTH - rxMsg.buf[0]; // Inverter X-akse
            yBall = rxMsg.buf[1];
        }

        // 4. Motta poengsum (Bare P2 reagerer)
        if (rxMsg.id == idGameOver && isPlayer2 && rxMsg.len == 2) {
            scoreP1 = rxMsg.buf[0];
            scoreP2 = rxMsg.buf[1];

            // NY: Sjekk om spillet er over
            if (scoreP1 >= WINNING_SCORE || scoreP2 >= WINNING_SCORE) {
                isGameOver = true;
                // IKKE pause her, la hovedloopen håndtere "Game Over"
            }
        }

        // 5. NY: Motta Reset-signal
        if (rxMsg.id == idResetGame && rxMsg.len == 1 && rxMsg.buf[0] == 1 ...
            && isGameOver) {
            // Den andre spilleren vil restarte
            Serial.println("Mottok reset-signal!");
            resetGame();
        }
    }
}

```

Listing 3: Sending av CAN meldinger

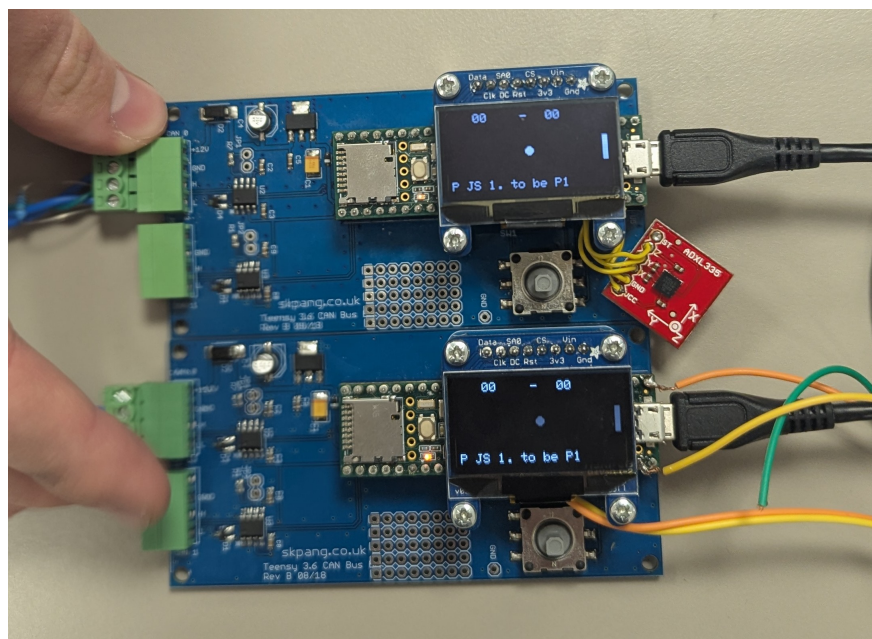
```

*/
void sendCANMessages() {
    if (isPlayerAssigned) {
        // 1. Send EGEN plateposisjon
        CAN_message_t msgPlatePosition;
        msgPlatePosition.id = isPlayer2 ? idPlatePositionP2 : ...
            idPlatePositionP1;
        msgPlatePosition.len = 1;
        msgPlatePosition.buf[0] = platePosition;
        Can0.write(msgPlatePosition);

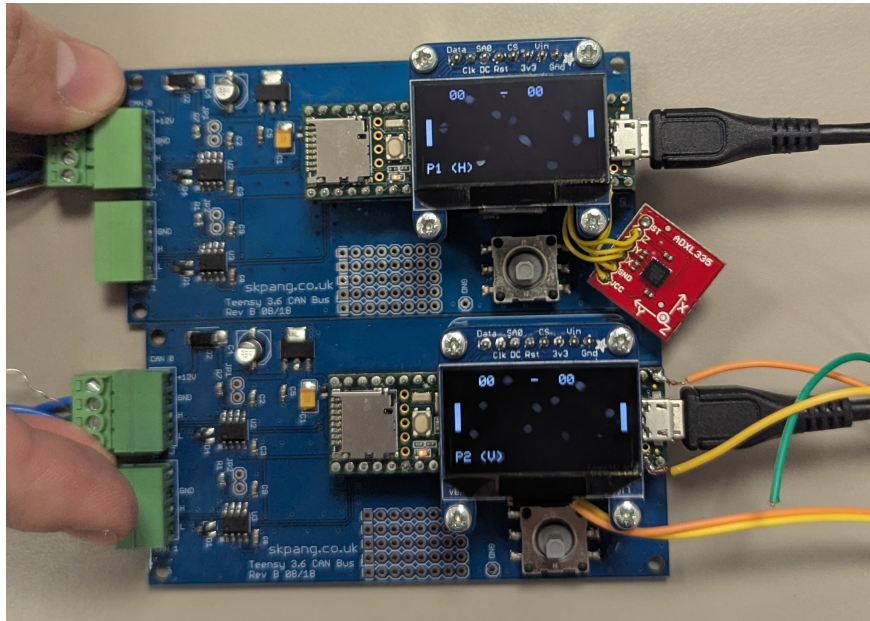
        // 2. Bare P1 sender ballposisjonen
        if (!isPlayer2) {
            CAN_message_t msgBall;
            msgBall.id = idBallPosition;
            msgBall.len = 2;
            msgBall.buf[0] = xBall;
            msgBall.buf[1] = yBall;
            Can0.write(msgBall);
        }
    }
}

```

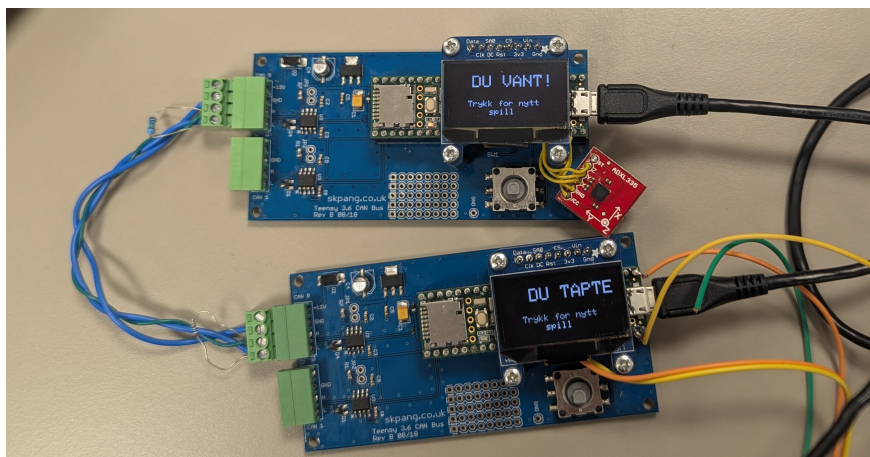
Listing 4: Motak av CAN meldinger



Figur 4.4: To Teensy-enheter som starter å spille mot hverandre



Figur 4.5: To Teensy-enheter midt i et spill. Det hvite som er på skjermene er ballen som driver å spretter rundt



Figur 4.6: To Teensy-enheter hvor en har vunnet og den andre har tapt

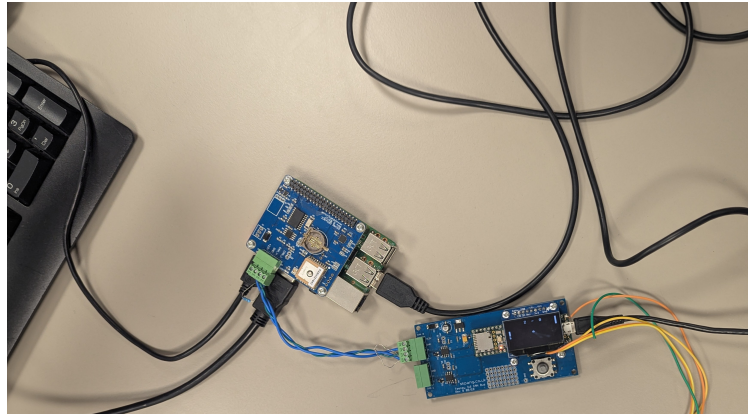
Til slutt ble det utviklet et operativsystem på en Raspberry Pi 3 ved hjelp av Buildroot. Systemet ble konfigurert med riktig ARMv8-A arkitektur, og et kjørbart program for Ping-Pong-spillet ble kompilert og inkludert i rotfilssystemet. Programmet implementerte all spilllogikken og var basert på samme prinsipper som Teensy-varianten, men tilpasset kjøring på en mikroprosessor med operativsystem.

For å teste løsningen ble Raspberry Pi koblet til en Teensy 3.6 via CAN-bus. Raspberry'en fungerte som én spiller med tastaturstyring, mens Teensyen ble brukt som den andre spilleren med joystick. Under testingen ble systemet kontinuerlig omkompilert og oppdatert for å validere at operativsystemet og spillapplikasjonen startet korrekt etter hver endring. Hver gang en endring ble gjort i spillkoden måtte det bli laget et nytt executable fil i operativsystemet. Dette ble gjort ved å bruke samme pc som operativsystem ble laget på, funnet buildroot-output/build/can-pingpong-3.14, og endret på .cpp filen som er i denne mappen. Etter det går man inn i terminalen og sjekker om det er en .o fil med å skrive `ls *.o`, for å så skrive en `rm *.o` for å fjerne denne filen. Deretter kan du skrive inn en `make` for å lage en ny executable fil som kan legges inn i operativsystemet med hjelp av `sudo scp can-pingpong / (path til hvor microSD kortet med operativsystemet ligger)`. Hele denne prosessen tar litt tid, men da slipper man å recompile hele operativsystemet igjen, hver gang en

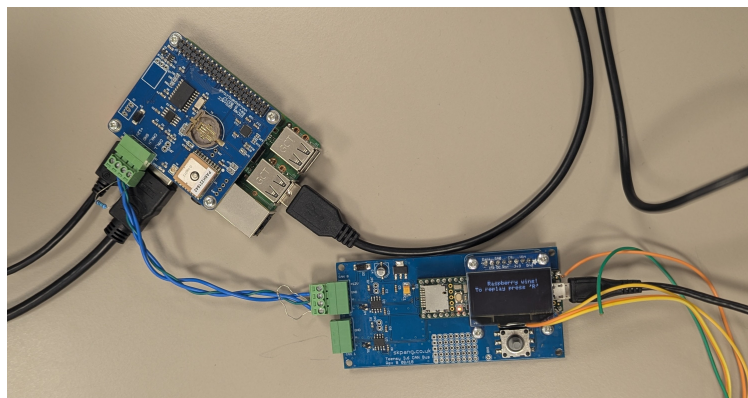
liten endring blir gjort i en fil.

Systemet viste stabile resultater: Raspberry Pi bootet uten problemer etter hver Buildroot-oppdatering, programmet startet konsekvent, og kommunikasjonen mellom Raspberry og Teensy via CAN-bus var stabil gjennom hele testperioden. Ball- og racketposisjoner ble synkronisert i sanntid uten merkbar forsinkelse.

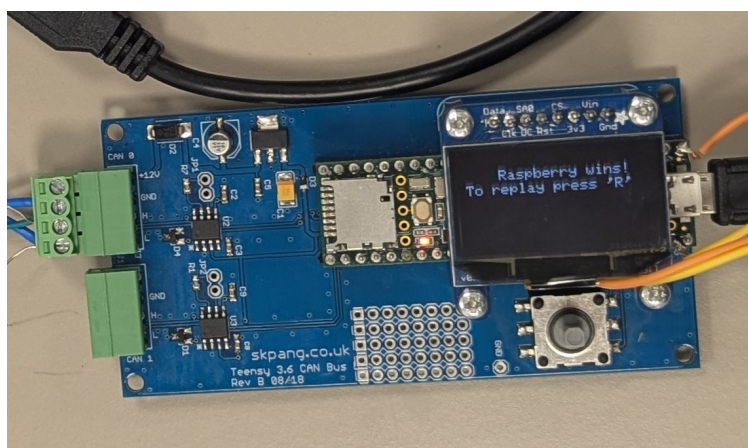
Det oppstod et mindre problem knyttet til reset-funksjonen, der trykknappen i joysticken kunne utløse reset ved et uhell. Dette ble løst ved å deaktivere joystick-reset og i stedet bruke tastaturet på Raspberry Pi, hvor «R»-tasten ble satt som manuell reset-funksjon. Etter denne justeringen fungerte systemet uten videre problemer.



Figur 4.7: Raspberry vs Teensy i Ping-Pong



Figur 4.8: Raspberry vs Teensy i Ping-Pong



Figur 4.9: Zoomed inn bilde av 4.9

Resultatene samlet sett demonstrerer en vellykket implementasjon av et system som kombinerer mikrokontroller, mikroprosessor og CAN-bus i et fungerende sanntids applikasjonsmiljø. Samlet sett viser resultatene at målene for prosjektet ble oppnådd, og at systemet kan utvides videre til mer komplekse applikasjoner.

Kapittel 5

Konklusjon

Gjennom prosjektet har gruppen fått en praktisk tilnærming til design og implementering av innebygde datasystemer. Arbeidet var fra grunnleggende mikrokontrollere til konfigurasjon av et eget operativsystem, som gav erfaring om samspillet mellom maskinvare og programvaren.

Arbeidet med ATmega168 ga erfaring med tegning av kretsskjema og valg av komponenter, som for eksempel spenningsregulator tilpasset kretsenens krav. koding på en slik enhet fikk gruppen til å fokusere på en mer effektiv kode siden det er begrenset med maskinkraft. Effektive løsninger som ferdig utregnede verdier gjorde at tung prosessering ble unngått. Dette gav en LED-diode som blinket mykt, og regelmessig.

Implementering av CAN-bus gav innblikk i virkemåten til denne kommunikasjonsprotokollen, og hvordan den kan benyttes til samkjøring av flere enheter. Dette ble brukt i to versjoner av et “ping-pong”-spill som ble kjørt både mellom to teensy 3.6 og mellom en teensy og Raspberrypi 3. Dette synliggjorde hvilke pakker og drivere som kreves for å få ulike enheter til å kommunisere sammen. Prosessen viste hvordan kryss-kommunikasjon nødvendigvis ikke er “plugg and play”, men kan kreve litt tilrettelegging av enhetene.

Det ble konfigurert et operativsystem til Raspberrypi 3, basert på et linux operativ system for ARM arkitektur. Det var egnet til å kjøre C++ kode, og kommunisere over ethernet, og CAN-bus til andre enheter. Samlet har disse oppgavene gitt en bedre forståelse for hvordan man må bygger opp et fungerende innebygde datasystem, og utvikler kode til det.

Gruppen lyktes dermed med hovedmålet om å bygge et fungerende innebygdsystem med egendefinert OS som styrer

, og hvilke problemer som medfølger

Tillegg A

SoftBlink kode

```
#define F_CPU 1000000UL

#include <avr/io.h>
#include <util/delay.h>
#include <math.h>
#include <avr/pgmspace.h>

#define LED_PIN PD6

// More aggressive gamma table (2.8 gamma) - spends less time at top brightness
const uint8_t gamma_table[64] PROGMEM = {
    0,  0,  0,  1,  1,  2,  2,
    3,  4,  5,  6,  7,  8,  10, 11,
    13, 15, 17, 19, 21, 23, 26, 28,
    31, 34, 37, 40, 43, 47, 50, 54,
    58, 62, 66, 70, 75, 79, 84, 89,
    94, 99, 104, 110, 115, 121, 127, 133,
    139, 145, 152, 158, 165, 172, 179, 186,
    193, 201, 208, 216, 224, 232, 240, 248
};

void setup_soft_blink(void) {
    TCCR0A = (1 << WGM00) | (1 << COM0A1);
    TCCR0B = (1 << CS01) | (1 << CS00);
    OCROA = 0;
    DDRD |= (1 << LED_PIN);
}

/*
// Prescaler = 1
TCCR0B |= (1 << CS00);

// Prescaler = 8
TCCR0B |= (1 << CS01);
*/

// Prescaler = 64
//TCCR0B |= (1 << CS01) | (1 << CS00);

// Prescaler = 256
//    TCCR0B |= (1 << CS02);
```

```

// Prescaler = 1024
//TCCR0B |= (1 << CS02) | (1 << CS00);

void set_gamma_corrected_brightness(uint8_t linear_level) {
    // Scale 0-255 to 0-63 and lookup
    uint8_t index = linear_level >> 2; // Divide by 4
    OCROA = pgm_read_byte(&gamma_table[index]);
}

void soft_blink_loop(void) {
    // Fade in
    for (uint16_t i = 0; i < 256; i++) {
        set_gamma_corrected_brightness(i);
        _delay_ms(56);
    }

    // Fade out
    for (uint16_t i = 255; i > 0; i--) {
        set_gamma_corrected_brightness(i);
        _delay_ms(56);
    }
}

int main(void) {
    setup_soft_blink();
    while(1) { soft_blink_loop(); }
}

```

Tillegg B

PING-PONG kode for Teensy 3.6

```
/*
=====
    TEENSY PONG - 2 spillere over CAN-bus med automatisk rollefordeling
=====
... (Variabeloversikt og CAN ID-er som før) ...
NY CAN ID: 58 (Reset Spill)
=====
*/

#include <Arduino.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <SPI.h>
#include <FlexCAN_T4.h>

// ----- Hardware -----
constexpr int JOY_RIGHT = 17;
constexpr int JOY_LEFT  = 18;
constexpr int JOY_CLICK = 19;
constexpr int JOY_UP    = 22;
constexpr int JOY_DOWN  = 23;

constexpr int OLED_DC      = 6;
constexpr int OLED_CS      = 10;
constexpr int OLED_RESET   = 5;
constexpr int SCREEN_WIDTH = 128;
constexpr int SCREEN_HEIGHT = 64;
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &SPI, OLED_DC, OLED_RESET, OLED_CS);

// ----- CAN-konfig -----
constexpr int groupNumber = 6;
constexpr int idPlatePositionP1 = groupNumber + 20; // 26
constexpr int idPlatePositionP2 = groupNumber + 21; // 27
constexpr int idBallPosition     = groupNumber + 50; // 56
constexpr int idRoleClaim        = groupNumber + 10; // 16
constexpr int idGameOver         = groupNumber + 51; // 57 (Sender poeng)
constexpr int idResetGame        = groupNumber + 52; // 58 (NY: Signal for å restarte)

FlexCAN_T4<CAN0, RX_SIZE_256, TX_SIZE_16> Can0;

// ----- Spillvariabler -----
```



```

// Plate
int platePosition = 22;
int remotePlatePosition = -1;
const int plateHeight = 20;
const int plateWidth = 4;

// Ball
int xBall = 64;
int yBall = 32;
int xVelocity = 1;
int yVelocity = 1;
const int ballRadius = 3;

// Poeng
int scoreP1 = 0; // P1 = Høyre spiller
int scoreP2 = 0; // P2 = Venstre spiller

// ----- Spilleets tilstand -----
bool isPlayer2 = false;
bool isPlayerAssigned = false;
bool hasClaimedRole = false;

// --- NYE VARIABLER FOR RESET ---
const int WINNING_SCORE = 5; // Spiller til 5 poeng
bool isGameOver = false; // Styrrer om spillet er over

// =====
// SETUP
// =====
void setup()
{
    Serial.begin(9600);
    delay(200);

    // Initialiser joystick
    pinMode(JOY_LEFT, INPUT_PULLUP);
    pinMode(JOY_RIGHT, INPUT_PULLUP);
    pinMode(JOY_UP, INPUT_PULLUP);
    pinMode(JOY_DOWN, INPUT_PULLUP);
    pinMode(JOY_CLICK, INPUT_PULLUP); // Viktig for reset

    // Initialiser skjerm
    if (!display.begin(SSD1306_SWITCHCAPVCC)) {
        Serial.println(F("ERROR: display.begin() failed."));
        while (true) delay(1000);
    }
    display.clearDisplay();
    display.setTextSize(1);
    display.setTextColor(SSD1306_WHITE);
    display.setCursor(30, 28);
    display.print("Venter...");
    display.display();

```



```

    // Initialiser CAN-bus
    Can0.begin();
    Can0.setBaudRate(500000);
    Serial.println("CAN-bus startet!");
}

// =====
// Funksjons-prototyper
// =====
bool handleJoystickInput();
void handleRoleAssignment(bool moved);
void receiveCANMessages();
void updateGameLogic();
void drawDisplay();
void sendCANMessages();
void drawGameOverScreen(); // NY
void resetGame();         // NY

// =====
// HOVEDLØKKE (LOOP)
// =====
void loop()
{
    if (isGameOver) {
        // --- SPILLET ER OVER ---

        // 1. Vis "Game Over"-skjermen
        drawGameOverScreen();

        // 2. Sjekk om DENNE spilleren vil restarte (trykker klikk)
        if (digitalRead(JOY_CLICK) == LOW) {
            // Send reset-signal til den andre spilleren
            CAN_message_t resetMsg;
            resetMsg.id = idResetGame;
            resetMsg.len = 1;
            resetMsg.buf[0] = 1; // Signal om å restarte
            Can0.write(resetMsg);

            // Vent til knappen slippes (viktig!)
            while(digitalRead(JOY_CLICK) == LOW) delay(10);

            // Utfør lokal reset
            resetGame();
        }

        // 3. Lytt etter reset-signal fra den ANDRE spilleren
        // (Må gjøres selv om vi venter på klikk)
        receiveCANMessages();
    } else {
        // --- VANLIG SPIL-LØKKE ---

        // 1. Les input fra spilleren
    }
}

```

```

    bool moved = handleJoystickInput();

    // 2. Håndter rollefordeling (skjer bare i starten)
    handleRoleAssignment(moved);

    // 3. Motta og behandle alle innkommende CAN-meldinger
    receiveCANMessages();

    // 4. Oppdater spill-logikk (KUN P1 gjør dette)
    if (!isPlayer2 && isPlayerAssigned) {
        updateGameLogic();
    }

    // 5. Tegn alt til skjermen
    drawDisplay();

    // 6. Send relevante CAN-meldinger
    sendCANMessages();
}

// 7. Kort pause for å stabilisere løkken
delay(10); // 100 Hz
}

// =====
// HJELPEFUNKSJONER (kalt fra loop)
// =====

/*
 * Leser joystick-input og oppdaterer 'platePosition'.
 */
bool handleJoystickInput() {
    bool moved = false;
    if (digitalRead(JOY_UP) == LOW && platePosition > 0) {
        platePosition -= 2;
        moved = true;
    }
    if (digitalRead(JOY_DOWN) == LOW && platePosition < SCREEN_HEIGHT - plateHeight) {
        platePosition += 2;
        moved = true;
    }
    return moved;
}

/*
 * Håndterer logikken for å bli P1 hvis man beveger seg først.
 */
void handleRoleAssignment(bool moved) {
    if (moved && !isPlayerAssigned && !hasClaimedRole) {
        isPlayer2 = false;
        isPlayerAssigned = true;
        hasClaimedRole = true;
        Serial.println("Jeg ble Player 1!");
    }
}

```

```

        CAN_message_t claimMsg;
        claimMsg.id = idRoleClaim;
        claimMsg.len = 1;
        claimMsg.buf[0] = 1;
        Can0.write(claimMsg);
    }
}

/*
 *Mottar og behandler alle CAN-meldinger i køen.
 */
void receiveCANMessages() {
    CAN_message_t rxMsg;
    while (Can0.read(rxMsg))
    {
        // 1. Rolleclaim
        if (rxMsg.id == idRoleClaim && rxMsg.len >= 1 && !isPlayerAssigned) {
            if (rxMsg.buf[0] == 1) {
                isPlayer2 = true;
                isPlayerAssigned = true;
                Serial.println("Jeg ble Player 2!");
            }
        }

        // 2. Plateposisjon fra motpart
        const int rxPlateId = isPlayer2 ? idPlatePositionP1 : idPlatePositionP2;
        if (rxMsg.id == rxPlateId && rxMsg.len >= 1) {
            remotePlatePosition = rxMsg.buf[0];
        }

        // 3. Ballposisjon (Bare P2 mottar)
        if (isPlayer2 && rxMsg.id == idBallPosition && rxMsg.len == 2) {
            xBall = SCREEN_WIDTH - rxMsg.buf[0]; // Inverter X-akse
            yBall = rxMsg.buf[1];
        }

        // 4. Motta poengsum (Bare P2 reagerer)
        if (rxMsg.id == idGameOver && isPlayer2 && rxMsg.len == 2) {
            scoreP1 = rxMsg.buf[0];
            scoreP2 = rxMsg.buf[1];

            // NY: Sjekk om spillet er over
            if (scoreP1 >= WINNING_SCORE || scoreP2 >= WINNING_SCORE) {
                isGameOver = true;
                // IKKE pause her, la hovedloopen håndtere "Game Over"
            }
        }

        // 5. NY: Motta Reset-signal
        if (rxMsg.id == idResetGame && rxMsg.len == 1 && rxMsg.buf[0] == 1 && isGameOver) {
            // Den andre spilleren vil restarte
            Serial.println("Mottok reset-signal!");
            resetGame();
        }
    }
}

```

```

    }
}

/*
 * Oppdaterer ballens posisjon, kollisjoner og poeng.
 * DENNE FUNKSJONEN KJØRES KUN AV P1.
 */
void updateGameLogic() {
    // Kollisjon- og bevegelseslogikk (som før)
    xBall += xVelocity;
    yBall += yVelocity;

    // Kollisjon høyre plate (P1, egen)
    int plateX = SCREEN_WIDTH - plateWidth;
    if (xBall + ballRadius >= plateX &&
        yBall >= platePosition &&
        yBall <= platePosition + plateHeight &&
        xVelocity > 0) {
        xVelocity = -xVelocity;
        xBall = plateX - ballRadius;
    }

    // Kollisjon venstre plate (P2, remote)
    if (remotePlatePosition >= 0) {
        int p2X = 0;
        int rp = remotePlatePosition;
        if ((xBall - ballRadius) <= (p2X + plateWidth) &&
            yBall >= rp && yBall <= rp + plateHeight &&
            xVelocity < 0) {
            xVelocity = -xVelocity;
            xBall = p2X + plateWidth + ballRadius;
        }
    }

    // Kollisjon vegger (topp/bunn)
    const int topWallY = 0;
    if (yBall - ballRadius <= topWallY && yVelocity < 0) {
        yVelocity = -yVelocity;
        yBall = topWallY + ballRadius;
    }
    else if (yBall + ballRadius >= SCREEN_HEIGHT && yVelocity > 0) {
        yVelocity = -yVelocity;
        yBall = SCREEN_HEIGHT - ballRadius;
    }

    // --- ENDRET: Poeng tildeles ---
    if (xBall - ballRadius > SCREEN_WIDTH || xBall + ballRadius < 0) {

        if (xBall - ballRadius > SCREEN_WIDTH) {
            scoreP2++; // P2 (venstre) scorer
        } else {
            scoreP1++; // P1 (høyre) scorer
        }
    }
}

```

```

        // Send ALLTID poengsum til P2
        CAN_message_t scoreMsg;
        scoreMsg.id = idGameOver;
        scoreMsg.len = 2;
        scoreMsg.buf[0] = scoreP1;
        scoreMsg.buf[1] = scoreP2;
        Can0.write(scoreMsg);

        // NY: Sjekk om spillet er VUNNET
        if (scoreP1 >= WINNING_SCORE || scoreP2 >= WINNING_SCORE) {
            isGameOver = true;
            // IKKE reset ballen, la loopen gå til "Game Over"-tilstand
        } else {
            // Spillet er IKKE over, bare reset for neste runde
            delay(2000); // Pause for at spillerne skal se
            xBall = SCREEN_WIDTH / 2;
            yBall = SCREEN_HEIGHT / 2;
            xVelocity = -1; // Serve alltid mot P2
            yVelocity = 1;
        }
    }
}

/*
 * Tegner hele skjermbildet på nytt (scoreboard, plater, ball).
 */
void drawDisplay() {
    display.clearDisplay();
    display.setTextColor(SSD1306_WHITE);

    // 1. Poengtavle
    display.setTextSize(1);
    display.setCursor(SCREEN_WIDTH/2 - 45, 0);
    if(scoreP2 < 10) display.print("0");
    display.print(scoreP2);
    display.setCursor(SCREEN_WIDTH/2 - 6, 0);
    display.print("-");
    display.setCursor(SCREEN_WIDTH/2 + 15, 0);
    if(scoreP1 < 10) display.print("0");
    display.print(scoreP1);

    // 2. Rolleindikator
    display.setTextSize(1);
    display.setCursor(0, SCREEN_HEIGHT - 8);
    if (!isPlayerAssigned) display.print("P JS 1. to be P1");
    else if (isPlayer2) display.print("P2 (V)");
    else display.print("P1 (H)");

    // 3. Plater
    if (remotePlatePosition >= 0)
        display.fillRect(0, remotePlatePosition, plateWidth, plateHeight, SSD1306_WHITE);
    display.fillRect(SCREEN_WIDTH - plateWidth, platePosition, plateWidth, plateHeight, SSD1306_WHITE);

    // 4. Ball

```

```

display.fillCircle(xBall, yBall, ballRadius, SSD1306_WHITE);

// 5. Vis alt
display.display();
}

/*
 * Sender CAN-meldinger for plateposisjon og (hvis P1) ballposisjon.
 */
void sendCANMessages() {
    if (isPlayerAssigned) {
        // 1. Send EGEN plateposisjon
        CAN_message_t msgPlatePosition;
        msgPlatePosition.id = isPlayer2 ? idPlatePositionP2 : idPlatePositionP1;
        msgPlatePosition.len = 1;
        msgPlatePosition.buf[0] = platePosition;
        Can0.write(msgPlatePosition);

        // 2. Bare P1 sender ballposisjonen
        if (!isPlayer2) {
            CAN_message_t msgBall;
            msgBall.id = idBallPosition;
            msgBall.len = 2;
            msgBall.buf[0] = xBall;
            msgBall.buf[1] = yBall;
            Can0.write(msgBall);
        }
    }
}

// =====
// NYE FUNKSJONER FOR GAME OVER / RESET
// =====

/*
 * Viser vinner/taper-skjerm basert på poengsum.
 */
void drawGameOverScreen() {
    display.clearDisplay();
    display.setTextColor(SSD1306_WHITE);

    // Sjekk hvem som vant
    bool jegVant = false;
    if (isPlayer2 && scoreP2 >= WINNING_SCORE) {
        jegVant = true;
    } else if (!isPlayer2 && scoreP1 >= WINNING_SCORE) {
        jegVant = true;
    }

    // Vis VANT eller TAPTE
    display.setTextSize(2);
    if (jegVant) {
        display.setCursor(25, 10);
        display.print("DU VANT!");
    }
}

```

```

    } else {
        display.setCursor(30, 10);
        display.print("DU TAPTE");
    }

    // Vis instruksjoner for reset
    display.setTextSize(1);
    display.setCursor(20, 40);
    display.print("Trykk for nytt");
    display.setCursor(45, 50);
    display.print("spill");

    display.display();
}

/*
 * Nullstiller alle spillvariabler for en ny runde.
 */
void resetGame() {
    Serial.println("Starter nytt spill...");

    // Reset poeng og tilstand
    scoreP1 = 0;
    scoreP2 = 0;
    isGameOver = false;

    // Reset posisjoner
    platePosition = 22;
    remotePlatePosition = -1;
    xBall = 64;
    yBall = 32;
    xVelocity = 1; // Start mot P1
    yVelocity = 1;

    // Gi en kort pause for å unngå "flimmer" eller race conditions
    delay(500);
}

```

Bibliography

- [1] *ATmega48/V / 88/V / 168/V*. PDF manual. Atmel. 2016. URL: [ATmega48_88_168_megaAVR-Data-Sheet-40002074.pdf](#).
- [2] *Atmel-ICE*. PDF Manual. Atmel. 2016. URL: [Atmel-ICE_UserGuide.pdf](#).
- [3] *AVR042: AVR Hardware Design Considerations*. PDF manual. Atmel. 2016. URL: [atmel-2521-avr-hardware-design-considerations_applicationnote_avr042.pdf](#).
- [4] *CAN - Controller Area Network Bus*. The European Space Agency. URL: https://www.esa.int/Enabling_Support/Space_Engineering_Technology/Onboard_Computers_and_Data_Handling/CAN_-_Controller_Area_Network_Bus.
- [5] CircuitBread. *Understanding CAN: A Beginner's Guide to the Controller Area Network Protocol*. 2020. URL: <https://www.circuitbread.com/tutorials/understanding-can-a-beginners-guide-to-the-controller-area-network-protocol> (sjekket 17.11.2025).
- [6] Bjørn B. Larsen. «mikrokontroller». I: *Store Norske Leksikon* (2025).
- [7] Ivar M. Liseter. «operativsystem». I: *Store Norske Leksikon* (2025).
- [8] *Luxorparts LED-sortiment 3 mm 100-pk*. Kjell & Compamny. URL: <https://www.kjell.com/no/produkter/elektro-og-verktoy/elektronikk/komponentsett/luxorparts-led-sortiment-3-mm-100-pk.-p90417>.
- [9] PJRC:Electronic Projects. *Teensy® 3.6 Development Board*. 2023. URL: <https://www.pjrc.com/store/teensy36.html> (sjekket 23.11.2025).
- [10] Halvor Bothner-By Ragnar Johnsen. «kommunikasjonsprotokoll». I: *Store Norske Leksikon* (2025).
- [11] Raspberry Pi Ltd. *Raspberry Pi 3 Model B+ Product Brief*. Raspberry Pi Ltd, 2025. URL: [/mnt/data/raspberry-pi-3-b-plus-product-brief.pdf](#) (sjekket 23.11.2025).
- [12] RaspberryPi. *Processors*. 2025. URL: <https://www.raspberrypi.com/documentation/computers/processors.html> (sjekket 23.11.2025).
- [13] RaspberryPi. *Raspberry Pi 3 Model B*. n/a. URL: <https://www.raspberrypi.com/products/raspberry-pi-3-model-b/> (sjekket 23.11.2025).
- [14] Eirik RossenKjell Bratbergsengen Tom Heine Nätt. «mikroprosessor». I: *Store Norske Leksikon* (2025).
- [15] *Very low drop voltage regulator with inhibit function*. PDF manual. STMicroelectronics. 2017. URL: <https://www.st.com/content/ccc/resource/technical/document/datasheet/c4/0e/7e/2a/be/bc/4c/bd/CD00000546.pdf/files/CD00000546.pdf/jcr:content/translations/en.CD00000546.pdf>.